CUBESATELLITE
C++/ARDUINO FRAMEWORK AND PYTHON CODING STANDARDS
FOR THE DEVELOPMENT AND DEMONSTRATION

Document ID: IAU-CSD-STD-001 Version: A
December 2025

Copyright 2025 by Group 1.

Can be used for other projects to practice programming standards.

Francis Mike John Camogao
AUTHOR/PREPARED BY

This page intentionally left blank

DISTRIBUTION AND LICENSE

This document is released under an **Open Access License**.

Permission is hereby granted, free of charge, to any person, educational institution, or organization to obtain a copy of this document and associated documentation files (the "Standards"), to deal in the Standards without restriction, including without limitation the rights to use, copy, modify, merge, publish, and distribute copies of the Standards, subject to the following conditions:

1. **Attribution:** The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Standards. Any derivative works, research papers, or project documentation utilizing these standards must explicitly cite the original author and institution.

2. **Citation Format:**

Camogao, F. M. J. (2025). *CubeSatellite Software Coding Standards & Guidelines (IAU-CSSD-STD-001)*. System Development Division, Indiana Aerospace University.

DISCLAIMER OF WARRANTY

THIS DOCUMENT AND THE STANDARDS CONTAINED HEREIN ARE PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT.

IN NO EVENT SHALL THE AUTHORS, THE INDIANA AEROSPACE UNIVERSITY, OR THE CUBESATELLITE PROJECT TEAM BE LIABLE FOR ANY CLAIM, DAMAGES, MISSION FAILURE, OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THESE STANDARDS OR THE USE OR OTHER DEALINGS IN THE STANDARDS.

TABLE OF CONTENTS

1. INTRODUCTION 6
    1.1 Purpose and Scope 6
    1.2 Core Philosophy 6
2. REFERENCED DOCUMENTS 7
3. GENERAL DESIGN 8
    3.1 Reliability and Portability 8
    3.2 Maintainability and Testability 8
    3.3 Reusability and Extensibility 8
4. VISUAL FORMATTING STANDARDS 9
    4.1 Columnar Alignment (Strict) 9
    4.2 Naming Conventions 10
5. CODING STANDARD (FRAMEWORK) 12
    5.1 Memory Management (The "Anti-Fragmentation" Rule) .. 12
    5.2 Recursion (NASA Power of Ten Rule 1) 13
    5.3 Concurrency & Interrupt Service Routines (ISRs) 14
6. ROBUSTNESS & DEFENSIVE CODING 15
    6.1 Bounded Loops (Watchdog Pattern) 15
    6.2 Floating Point Safety 15
7. GROUND STATION STANDARDS (PYTHON) 16
    7.1 Concurrency & I/O (Threading) 16
    7.2 Data Logging (The "Black Box") 16
    7.3 Formatting (Configuration Blocks) 17
    7.4 Virtual Environments 17
8. CHECKLIST 18

Table 1. Change Log

Revision ID	Document Date	Change Authority	Affected Paragraphs	Comments
001 v.A	Dec 2025	F. Camogao	All	Original

1 INTRODUCTION

The intent of this document is to provide directions and guidance to C++ programmers and Python developers that will enable them to employ good programming style and proven programming practices leading to safe, reliable, testable, and maintainable code.

As indicated above, the Cube Satellite onboard computer and software or any related component, will be developed in C++ and Python. C++ was designed to support data abstraction, object-oriented programming, and generic programming while retaining compatibility with traditional C programming techniques. While Python was made driven by a simple goal: to make programming more intuitive and enjoyable. For this reason, the Cube Satellite systems Coding Standard shall and will focus on the following:

1. The Power of 10: Rules for Developing Safety-Critical Code by NASA

2. Vehicle Systems Safety Critical Coding Standards for C, and

3. C++ language-specific guidelines and standards.

The Power of 10 is used mainly because it dictates how a reliable software system should be. The rules are intended to eliminate certain coding practices that make a code difficult to review or statically analyze. It can also be implemented in various programming languages to serve as a guide, not just for one programming language in general.

This standard is developed **solely for this project**. However, you can use it to set your coding standards and coding style, or it can also serve to tell your AI (Personal Context), such as ChatGPT or Gemini, or even BlackBox, etc., to follow instructions, especially when providing a code snippet—best, you can also tell your AI coding assistant follow the reference standards for the best result.

CORE PHILOSOPHY

Our mission is Safety-Critical. A runtime error in orbit results in mission failure.

- **Defensive Programming:** Assume sensors will fail and data will be corrupted. Handle these cases explicitly.
- **Determinism:** Predictable execution time and memory usage are paramount.
- **Visual Clarity:** Code formatting must expose errors at a glance.

2 REFERENCED DOCUMENTS

These can also be used to develop your own coding style, since all of these are used to be the bases to draft this standard.

1. NASA Power of Ten
2. JSF AV C++
3. MISRA C++
4. PEP 8

Used with permission supplied via the following statement:

Permission is granted to any individual or institution to use, copy, modify, and distribute this document, provided that this complete copyright and permission notice is maintained intact in all copies.

3 GENERAL DESIGN

The purpose of this standard document is intended to help the onboard computer developers and other team members who might or may involved in the process of coding to achieve their respective team objectives. This helps the programmers to develop code that conforms to safety-critical software principles, i.e., code that does not contain defects that could lead to catastrophic failures resulting in significant harm to individuals and/or equipment. In general, the code produced should exhibit the following important qualities:

Reliability: Executable code should consistently fulfill all requirements in a predictable manner.

Portability: Source code should be portable (i.e. not compiler or linker dependent).

Maintainability: Source code should be written in a manner that is consistent, readable, simple in design, and easy to debug.

Testability: Source code should be written to facilitate testability. Minimizing the following characteristics for each software module will facilitate a more testable and maintainable module:

1. **code size**
2. **Complexity**
3. **static path count** (number of paths through a piece of code)

Reusability: The design of reusable components is encouraged. Component reuse can eliminate redundant development and test activities (i.e. reduce costs).

Extensibility: Requirements are expected to evolve over the life of a product. Thus, a system should be developed in an extensible manner (i.e. perturbations in requirements may be managed through local extensions rather than wholesale modifications).

Readability: Source code should be written in a manner that is easy to read, understand and comprehend.

Note that following the guidelines contained within this document will not guarantee the production of an error-free, safe product. However, adherence to these guidelines, as well as the processes defined in the Software Development Plan [12], will help programmers produce clean designs that minimize common sources of mistakes and errors.

** The following texts are copied from the JSF-AV Rules PDF since it still applicable to our approach.*

4 VISUAL FORMATTING STANDARDS (STRICT)

Severity: MANDATORY. Code that is not aligned will be rejected during the review process.

4.1 Columnar Alignment

4.1.1 Rule: Variable declarations, constants, and assignments within the same scope must be vertically aligned into columns.

Structure:

[Modifier] -> [Type] -> [Name] -> [Operator] -> [Value]

4.1.2 Rationale: Ensures missing modifiers (like const) or type mismatches are immediately visible to the Supervisor.

Example 4.1: Variable Declaration

C++ (incorrect)

```
const int16_t servo_min = 1000;
const extern uint32_t baud_rate = 9600;
float battery_voltage = 0.0f;
volatile bool is_active = false;
```

C++ (correct)

```
// Modifier        Type         Variable Name      Operator  Value
const              int16_t      SERVO_MIN_US       =         1000;
const extern       uint32_t     BAUD_RATE          =         9600;
                   float        battery_volt_v     =         0.0f;
volatile           bool         is_system_active   =         false;
```

Left blank intentionally.

4.2 Naming Conventions

4.2.1 Variables: snake_case with Physical Units.

4.2.2 Constants: UPPER_SNAKE_CASE.

4.2.3 Functions: PascalCase.

Example 4.2.1: Variable (snake_case)

C++ (incorrect)

```cpp
// Bad: What is 'val'? Is 'time' seconds or ms?
float dist = 100.5;
uint32_t time = 5000;
int16_t val = 340;
bool status = true;
```

C++ (correct)

```cpp
// Modifier        Type          Variable Name      Operator  Value
                   float         distance_m         =         100.5f;
                   uint32_t      timeout_ms         =         5000;
                   int16_t       sensor_val_raw     =         340;
volatile           bool          is_module_active   =         true;
```

Example 4.2.2: Constants (UPPER_SNAKE_CASE)

C++ (incorrect)

```cpp
const float pi = 3.14159;
const int max_temp = 85;
const int baud = 9600;
```

C++ (correct)

```cpp
// Modifier        Type          Constant Name      Operator  Value
const              float         PI_VAL             =         3.14159f;
const              int8_t        MAX_TEMP_CELSIUS   =         85;
const              uint32_t      TELEMETRY_BAUD     =         9600;
```

Example 4.2.3: Function (PascalCase) is next page.

Example 4.2.3: Functions (PascalCase)

C++ (incorrect)

```cpp
// Bad: Is 'check' a variable or a function?
bool check() { ... }
void run() { ... }
float get() { ... }
```

C++ (correct)

```cpp
// Good: Clear Action + Target
bool     IsSystemReady() { ... }
void     DeployAntenna() { ... }
float    GetBatteryVoltage() { ... }
```

Example: Putting it together (Contextual Example)

C++ (correct)

```cpp
void MonitorBatteryHealth() {
    // Modifier    Type       Name                Operator  Value
    const          float      MIN_VOLTAGE_V       =         3.2f;
    const          float      CRITICAL_LEVEL_V    =         3.0f;
                   float      current_volt_v      =         0.0f;

    // 1. Get Data (PascalCase Function)
    current_volt_v = GetBatteryReading();

    // 2. Logic Check (snake_case variable vs UPPER_CASE constant)
    if (current_volt_v < CRITICAL_LEVEL_V) {
        EnterSafeMode();
    }
    else if (current_volt_v < MIN_VOLTAGE_V) {
        TransmitLowBatteryWarning();
    }
}
```

Left blank intentionally.

5 CODING STANDARD (Framework)

5.1 Memory management (The "Anti-Fragmentation" Rule)

5.1.1 Rule: Dynamic memory allocation (*malloc*, *free*, *new*, *delete*) is **STRICTLY FORBIDDEN** after system initialization.

5.1.2 Specific Ban: The Arduino "*String*" class is banned.

5.1.3 Rationale: Dynamic allocation on embedded systems causes heap fragmentation. Over hours/day during operation, this will cause the OBC to crash or overwrite variables.

5.1.4 Standard: Use static arrays or the stack.

Example 5.1: Handling Text Data

C++ (incorrect)

```
// Bad: Uses dynamic memory. Fragmentation risk high.
String telemetry = "Alt: " + String(altitude) + ", Temp: " + String(temp);
```

C++ (correct)

```
// Good: Static allocation. Deterministic memory usage.
char telemetry_buffer[64];
snprintf(telemetry_buffer, sizeof(telemetry_buffer), "Alt: %d, Temp: %.2f", altitude, temp);
```

Left blank intentionally.

5.2 Recursion (NASA Power of Ten - Rule 1)

5.2.1 Rule: Functions must **NEVER** call themselves, directly or indirectly.

5.2.2 Rationale: Recursion makes stack usage unpredictable. A stack overflow in space writes over memory, causing a "Hard Fault".

5.2.3 Standard: Use loops (iteration) for all repetitive algorithms.

Example 5.2: Math Calculation

C++ (incorrect)

```cpp
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1); // RECURSION
}
```

C++ (correct)

```cpp
int factorial(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}
```

Left blank intentionally.

5.3 Concurrency & Interrupt Service Routines (ISRs)

5.3.1 Rule: ISRs must be "Atomic" (extremely short).

5.3.2 Strict Prohibition: NEVER put *Serial.print()*, *delay()*, or I2C calls inside an ISR.

5.3.3 Rationale: *Serial.print* relies on interrupts. Calling it inside an interrupt causes a "Deadlock" (the CPU freezes waiting for an interrupt that can never happen).

Example 5.3: Sensor Trigger

C++ (incorrect)

```cpp
void onSensorTrigger() {
    Serial.println("Sensor Triggered!"); // DEADLOCK RISK
    delay(10); // BLOCKS CPU
}
```

C++ (correct)

```cpp
// 1. Declare flag as volatile (tells compiler it can change anytime)
volatile bool sensor_triggered = false;

// 2. ISR only sets the flag
void onSensorTrigger() {
    sensor_triggered = true;
}

// 3. Handle the logic in the main loop
void loop() {
    if (sensor_triggered) {
        noInterrupts(); // Briefly pause interrupts to read safely
        sensor_triggered = false;
        interrupts();   // Re-enable interrupts

        Serial.println("Sensor Triggered!"); // Safe to print here
    }
}
```

Left blank intentionally.

6 ROBUSTNESS & DEFENSIVE CODING

6.1 Bounded Loops (Watchdog Pattern)

6.1.1 Rule: Every loop waiting for an external event must have a timeout.

Example 6.1: Waiting for Radio

C++ (incorrect)

```
while (!Radio.available()); // If radio breaks, satellite dies.
```

C++ (correct)

```
const          uint32_t     TIMEOUT_MS       =         100;
               uint32_t     start_time       =         millis();

while (!Radio.available()) {
    // Watchdog Check (Current Time - Start Time)
    if (millis() - start_time > TIMEOUT_MS) {
        log_error(ERR_RADIO_TIMEOUT);
        break;
    }
}
```

6.2 Floating Point Safety

6.2.1 Rule: Floating point equality (==) is forbidden. Use Epsilon comparisons.

Example 6.2: Voltage Check

C++ (incorrect)

```
if (voltage == 3.3) { ... } // Fails due to precision
```

C++ (correct)

```
const          float        EPSILON          =         0.001f;
if (abs(voltage - 3.3f) < EPSILON) { ... }
```

7 GROUND STATION STANDARDS (Python)

7.1 Concurrency & I/O (Threading)

7.1.1 Rule: Serial listening must occur in a separate thread (Non-blocking). All Serial I/O must be wrapped in `try-except` blocks to handle cable disconnections or driver failures.

7.1.2 Rationale: Reading from a radio transceiver is slow. If you put `serial.readline()` in your main GUI loop, the interface will freeze.

7.1.3 Standard: Use the `threading` module. Run the "Serial Listener" in a separate daemon thread.

Example 7.1: Serial Reading

Python (incorrect)

```python
# Freezes GUI, crashes on disconnect
line = serial.readline()
```

Python (correct)

```python
try:
    if serial_conn.in_waiting:
        line = serial_conn.readline().decode('utf-8')
except serial.SerialException:
    print("Connection Lost")
```

7.2 Data Logging (The "Black Box"

7.2.1 Rule: Log RAW data to CSV before parsing. Never discard data because it failed a checksum check (store it as "Bad Packet" for post-analysis). **Raw Data First.**

7.2.2 Rationale: If your parsing logic has a bug, you might delete valuable data by accident.

7.2.3 Standard: Log the **raw string** received from the satellite to a `.txt` or `.csv` file *before* attempting to parse or convert it.

7.3 Formatting

7.3.1 Rule: Configuration blocks must use **Columnar Alignment.**

Example 7.3: Formatting

Python (correct)

```python
# Configuration Block
SERIAL_PORT          = "COM3"
BAUD_RATE            = 9600
LOG_DIRECTORY        = "./mission_logs"
CSV_HEADERS          = ["Time", "Alt_m", "Temp_C", "Volt_V"]
```

7.4 Virtual Environments

7.4.1 Rule: Isolation is Mandatory.

7.4.2 Standard: The project must include a `requirements.txt` file.

7.4.2.1 Command: `pip freeze > requirements.txt`

7.4.3 Rationale: Ensures every team member uses the exact same version of `pyserial` and `matplotlib`.

Addendum: All Coding standards and styles from the C++ framework must also be applied to the Python framework.

Left blank intentionally.

8 CHECKLIST

1. **Visual Scan:** Are all variables aligned in columns? (Ref 4.1)
2. **Memory Check:** Is `String` used anywhere? (Ref 5.1) -> *Reject if YES.*
3. **Recursion Check:** Is any function calling itself? (Ref 5.2) -> *Reject if YES.*
4. **ISR Check:** Is `Serial.print` or `delay` used inside an ISR? (Ref 5.3) -> *Reject if YES.*
5. **Watchdog Check:** Does every `while` loop have a timeout counter? (Ref 6.1) -> *Reject if NO.*
6. **Unit Check:** Do all physical variables have units (e.g., `_m`, `_s`)? (Ref 4.2) -> *Reject if NO.*